

KINGDOM OF MOROCCO

Ministry of Higher Education

Engineering Sciences - Computer / Software Engineering

FINAL YEAR ENGINEERING PROJECT (PFE)

MyHeath

Telemedicine & connected health platform

ASP.NET Core 8 · React · MongoDB · Docker

Author / Student: Nezha Fekoussa

Academic year 2025 - 2026

Technical report for the examination board

Resume / Abstract

Resume (FR)

MyHeath est une plateforme de telemedecine (sante feminine et masculine) avec suivi quotidien, diabete, cycle, messagerie chiffree, coach IA et administration. Ce rapport presente le backend ASP.NET Core 8 realise par Nezha Fekoussa, avec React et MongoDB, Docker et deployment cloud.

Abstract (EN)

MyHeath is a telemedicine platform for women's and men's health tracking, diabetes care, encrypted messaging, AI coaching and admin subscriptions. This report documents the ASP.NET Core 8 backend authored by Nezha Fekoussa, keeping React and MongoDB, with Docker and cloud deployment guidance.

Keywords / Mots-cles

ASP.NET Core 8, React, MongoDB, Docker, JWT, AES-256, telemedicine, Claude AI

Table of contents

1. Introduction and context	5
1.1 Problem statement	5
1.2 Project objectives	5
1.3 Why ASP.NET Core	5
2. Requirements analysis	6
2.1 Actors	6
2.2 Functional requirements	6
2.3 Non-functional requirements	6
3. Technical architecture	7
3.1 Overview	7
3.2 Stack	7
3.3 Project structure	7
3.4 Node compatibility	8
4. Detailed design	9
4.1 MongoDB model	9
4.2 AES-256-CBC	9
4.3 Predictive analysis	10
5. .NET backend implementation	11
5.1 Authentication	11
5.2 Modules	11
5.3 Enhancements	12
6. React frontend	15
7. Docker and deployment	16
7.1 Containerization	16
7.2 Environment variables	16
7.3 Cloud	16
8. Testing and demo accounts	17
8.1 Validation scenarios	17
9. Node.js vs ASP.NET Core	18
10. Security	19

11. Conclusion	20
11.1 Deliverables	20
Appendix A - Sample requests	21
Appendix B - Docker checklist	22
Appendix C - Glossary	23

1. Introduction and context

1.1 Problem statement

Continuous health tracking is often fragmented. MyHeath unifies the patient-doctor-admin journey behind a secure API and a modern dashboard UI.

1.2 Project objectives

- Professional REST API with ASP.NET Core 8
- Keep the existing React frontend (/api contract)
- MongoDB persistence + AES-256-CBC + JWT
- Appointments, medication reminders, notifications, admin audit
- Document Docker and cloud deployment

1.3 Why ASP.NET Core

ASP.NET Core 8 matches taught C# / Web API / DI / JWT skills. This deliverable is authored by Nezha Fekoussa.

2. Requirements analysis

2.1 Actors

Actor	Main responsibilities
Patient	Tracking, diabetes, AI, appointm
Doctor	Consultations, appointments, mes
Administrator	Users, plans, subscriptions, aud

2.2 Functional requirements

- Auth + RBAC patient | doctor | admin
- Daily tracking + 0-100 score, periods, diabetes
- Encrypted records/chat, Claude AI Coach
- Subscriptions FREE / CARE / PREMIUM

2.3 Non-functional requirements

- JWT / BCrypt / AES security, Docker, Swagger

Figure 2.3 — Diagramme de cas d'utilisation

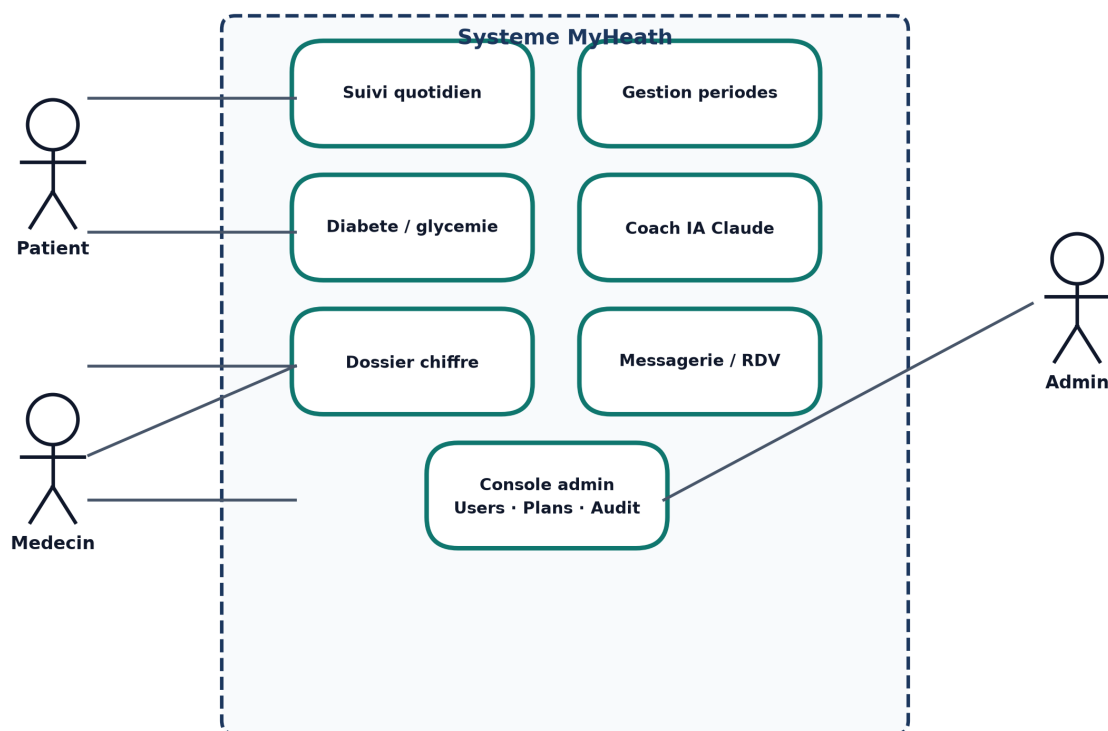


Figure 2.3 - Use case diagram

3. Technical architecture

3.1 Overview

3-tier architecture: React -> ASP.NET Core -> MongoDB (VITE_API_URL).

Figure 2.1 — Architecture 3-tiers MyHeath (.NET + React + MongoDB)

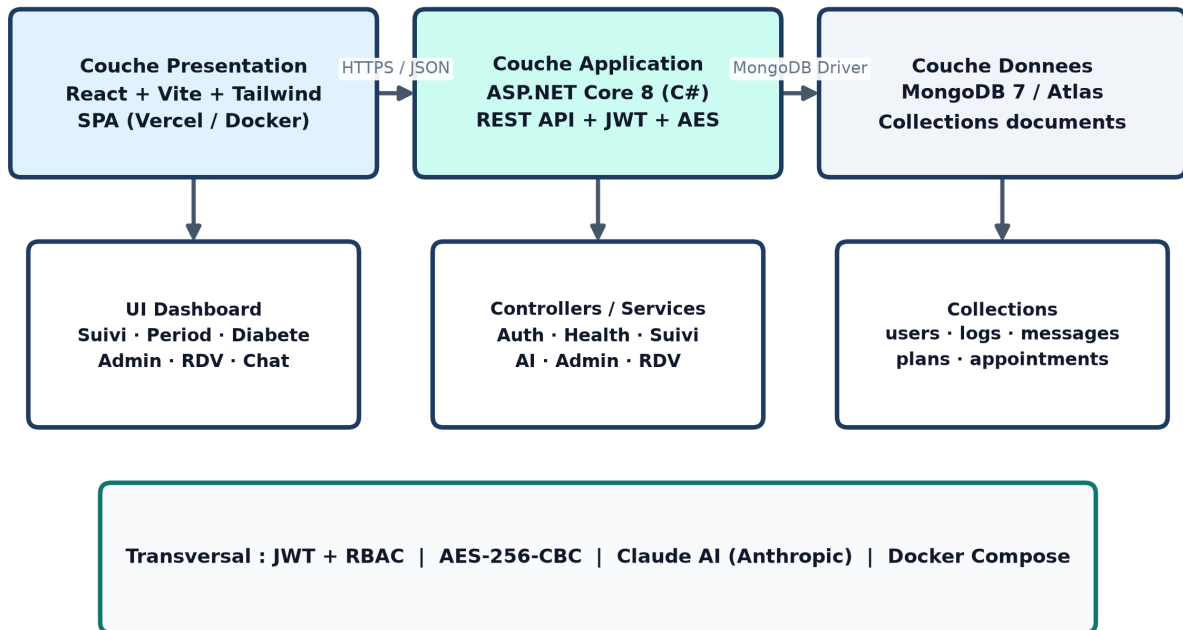


Figure 2.1 - MyHeath 3-tier architecture

3.2 Stack

Layer	Technologies
Backend	ASP.NET Core 8, C#, MongoDB.Driv
Frontend	React 18, Vite, Tailwind, Axios
Data	MongoDB 7 / Atlas
Ops	Docker Compose, Vercel, Azure

3.3 Project structure

```
backend-dotnet/MyHeath.Api/  
  Controllers/ Services/ Models/ Program.cs  
docker-compose.dotnet.yml
```

3.4 Node compatibility

Same /api/* contract; switch Node:5000 or .NET:5080 via VITE_API_URL.

4. Detailed design

4.1 MongoDB model

- users, healthrecords, symptomlogs, dailyhealthlogs, messages
- subscriptionplans, appointments, notifications, auditlogs

Figure 2.6 — Modele de domaines (extrait)

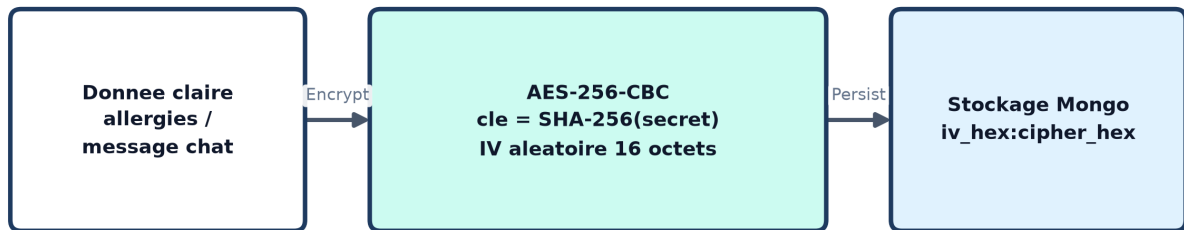


Figure 2.6 - Domain model

4.2 AES-256-CBC

Sensitive fields and messages encrypted (iv_hex:ciphertext_hex, SHA-256 key).

Figure 2.7 — Chiffrement AES-256-CBC des donnees sensibles



Dechiffrement uniquement cote API apres JWT + RBAC

Figure 2.7 - AES-256-CBC encryption

4.3 Predictive analysis

HealthScoreService (0-100) and CycleAnalyzer (phase, ovulation, anomalies).

Figure 2.8 — Pipeline score sante & analyse de cycle

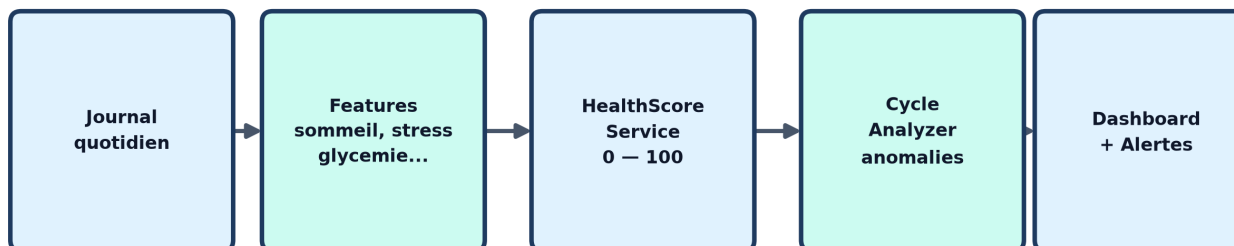


Figure 2.8 - Predictive pipeline

5. .NET backend implementation

5.1 Authentication

BCrypt cost 12, JWT HMAC-SHA256 for 7 days, [Authorize(Roles=...)].

Figure 2.4 — Sequence : authentication JWT

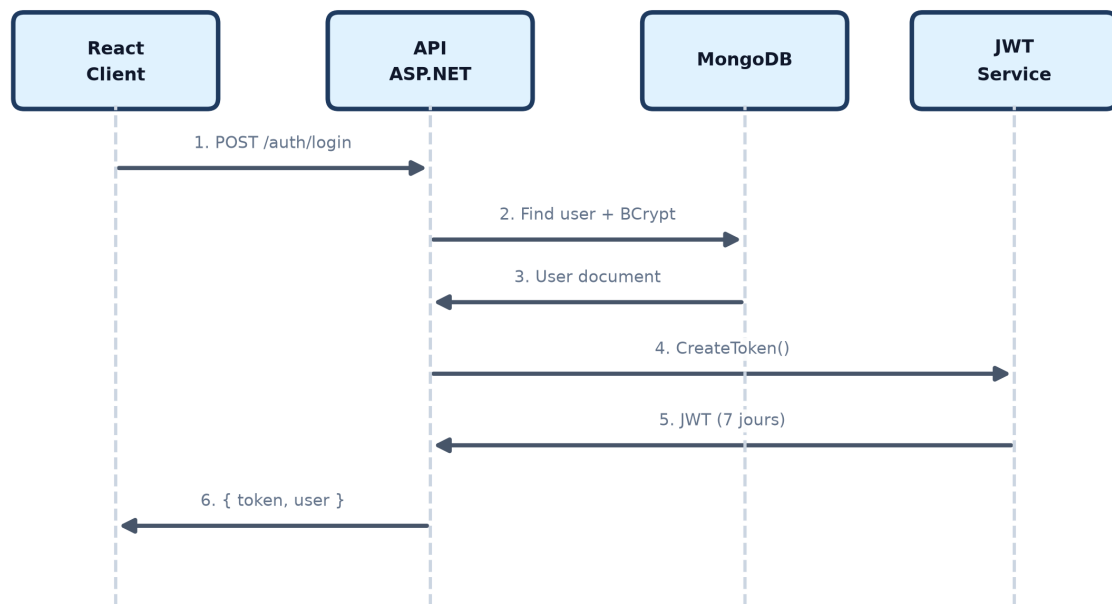
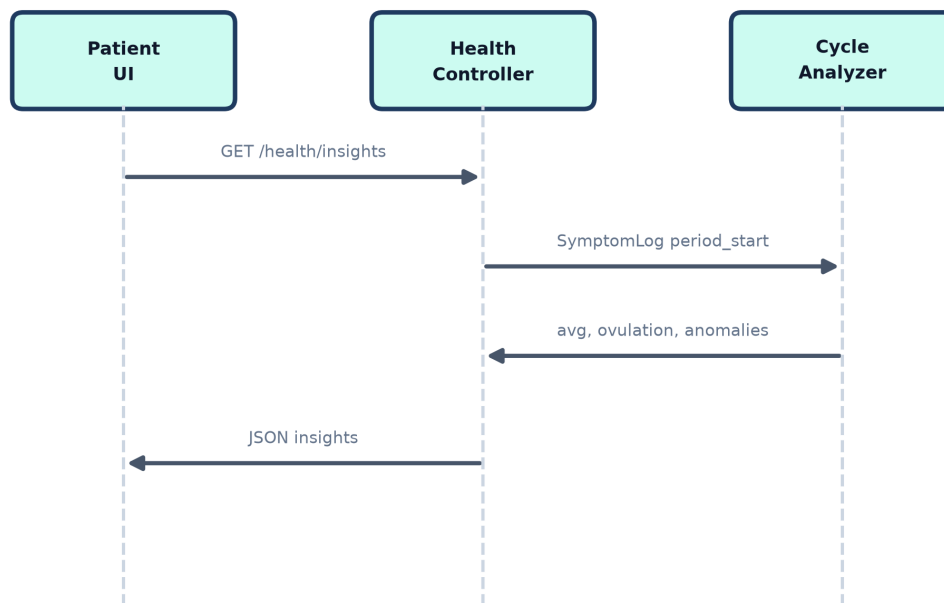


Figure 2.4 - Authentication sequence

5.2 Modules

- Health, Suivi, Chat, AI, Admin, Appointments, Medications, Notifications

Figure 2.5 — Sequence : calcul des insights cycle*Figure 2.5 - Insights sequence*

5.3 Enhancements

Feature	Endpoint
Appointments	/api/appointments
Notifications	/api/notifications
Medications	/api/medications
Audit	/api/admin/audit
Swagger	/swagger

Figure 3.1 — Organisation du backend ASP.NET Core

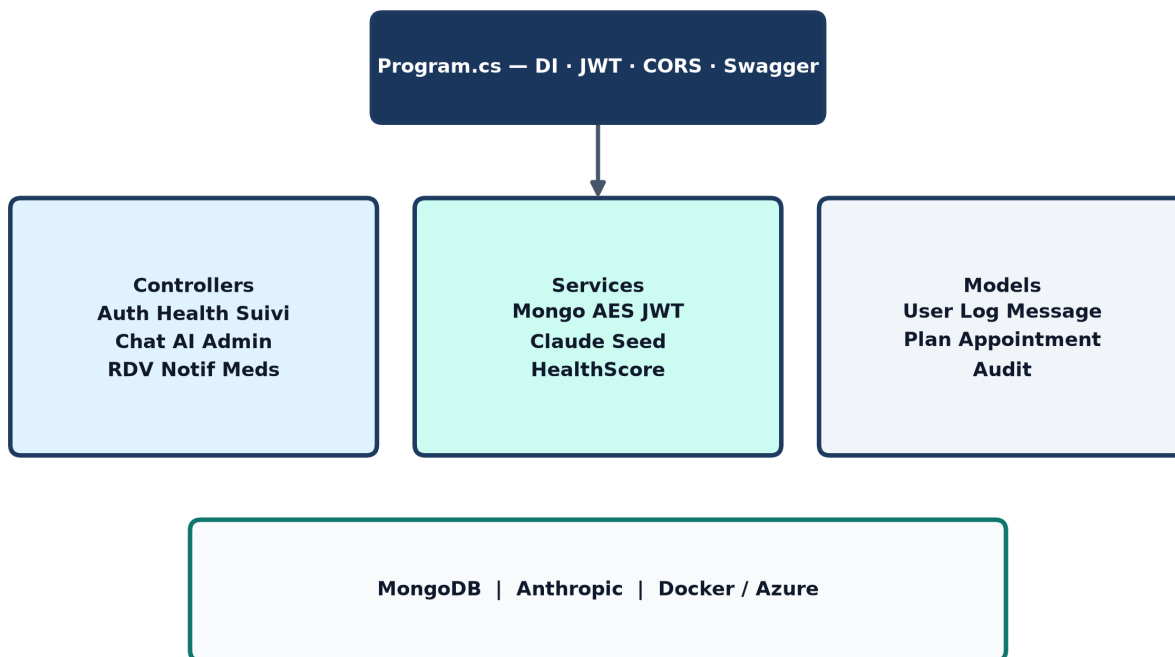


Figure 3.1 - Backend organization

Figure 3.3 — Cartographie des routes API (/api)

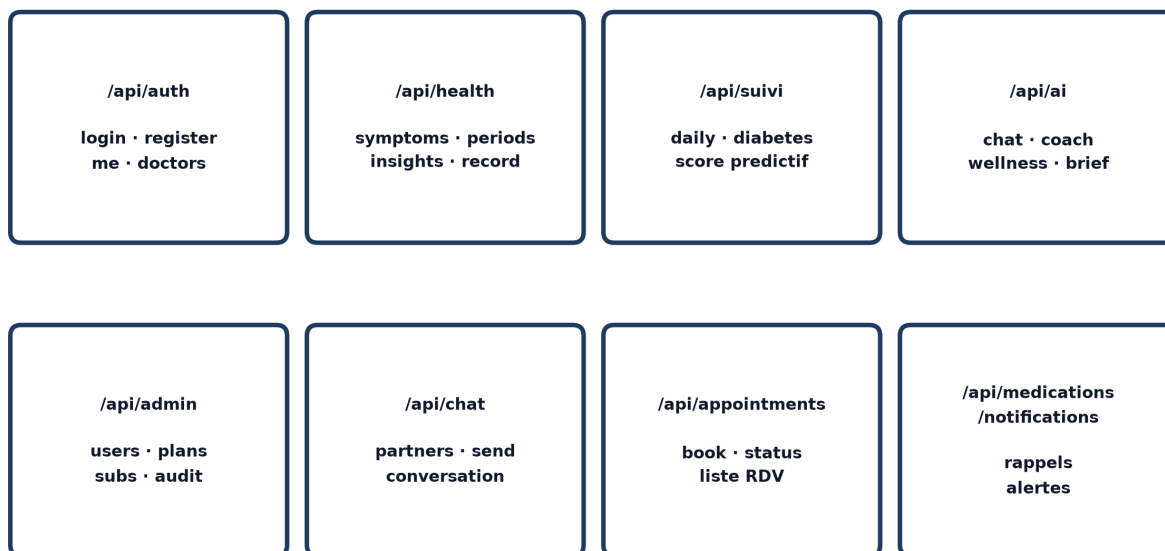
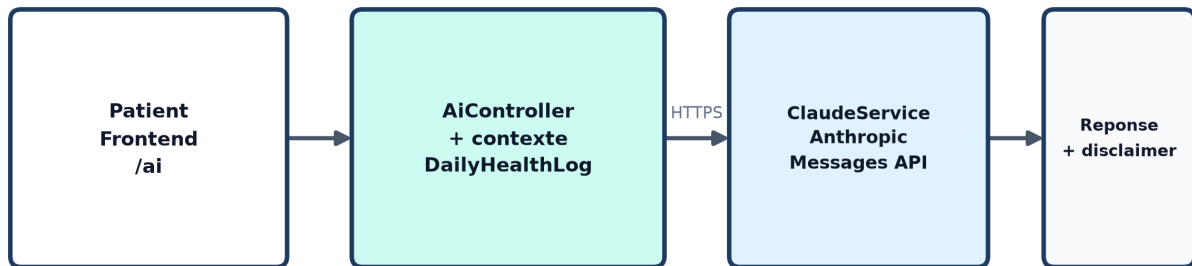


Figure 3.3 - API routes

Figure 2.9 — Integration MyHeath AI Coach (Claude)



Usage educatif uniquement — pas un diagnostic medical

Figure 2.9 - Claude AI Coach

6. React frontend

Sidebar dashboard: Admin, Suivi, Period, Diabetes, Medications, AI, Appointments, Chat.

- Deliverable author: Nezha Fekoussa

7. Docker and deployment

7.1 Containerization

```
docker compose -f docker-compose.dotnet.yml up --build
# API http://localhost:5080/api/healthcheck
# UI http://localhost:5173
```

Figure 2.2 — Topologie de déploiement (Docker / Cloud)

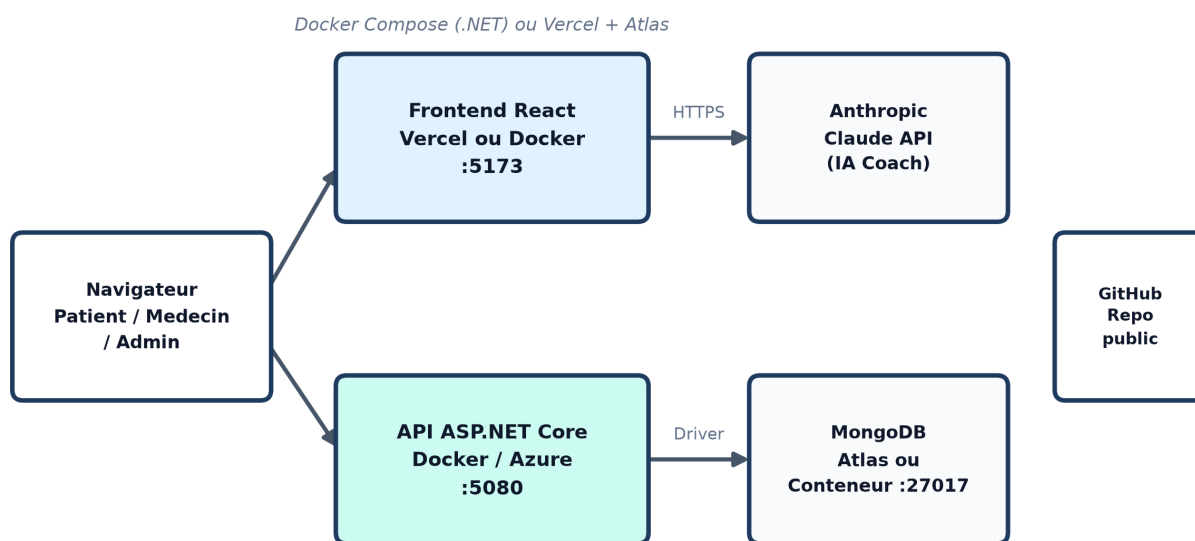


Figure 2.2 - Deployment topology

7.2 Environment variables

- MONGODB_URI, JWT_SECRET, AES_SECRET_KEY, CLIENT_URL, ANTHROPIC_API_KEY

7.3 Cloud

Frontend on Vercel, API via Docker/Azure, MongoDB Atlas. Never commit secrets.

8. Testing and demo accounts

Role	Email	Password
Admin	admin@myheath.app	Admin123
Doctor	doctor@myheath.app	Doctor123
Patient	patient@myheath.app	Patient123

8.1 Validation scenarios

- Admin subscriptions · Patient tracking/appointments · Encrypted chat · AI Coach

9. Node.js vs ASP.NET Core

Criterion	Node.js	ASP.NET Core
Language	JavaScript	C#
API docs	Manual	Swagger
PFE fit	MERN	Taught .NET stack

10. Security

- HTTPS, env secrets, CORS, RBAC, AES, AI non-diagnostic disclaimer

Aligned with Moroccan Law 09-08 / GDPR good practices.

11. Conclusion

This PFE demonstrates a telemedicine ASP.NET Core 8 API authored by Nezha Fekoussa, with React, MongoDB, Docker and cloud deployment. Next steps: SignalR, payments, mobile.

11.1 Deliverables

- GitHub: <https://github.com/salahEddine-Admou/MyHeath>
- GUIDE_INSTALLATION.md · docs/RAPPORT_PFE_MYHEATH_DOTNET_EN.pdf
- Author: Nezha Fekoussa

Appendix A - Sample requests

```
POST /api/auth/login
```

```
{ "email": "patient@myheath.app", "password": "Patient123" }
```

Appendix B - Docker checklist

- Docker Desktop · clone · compose up --build · healthcheck · login

Appendix C - Glossary

- JWT: JSON Web Token
- AES: Symmetric encryption
- RBAC: Role-based access control
- Swagger: OpenAPI documentation